

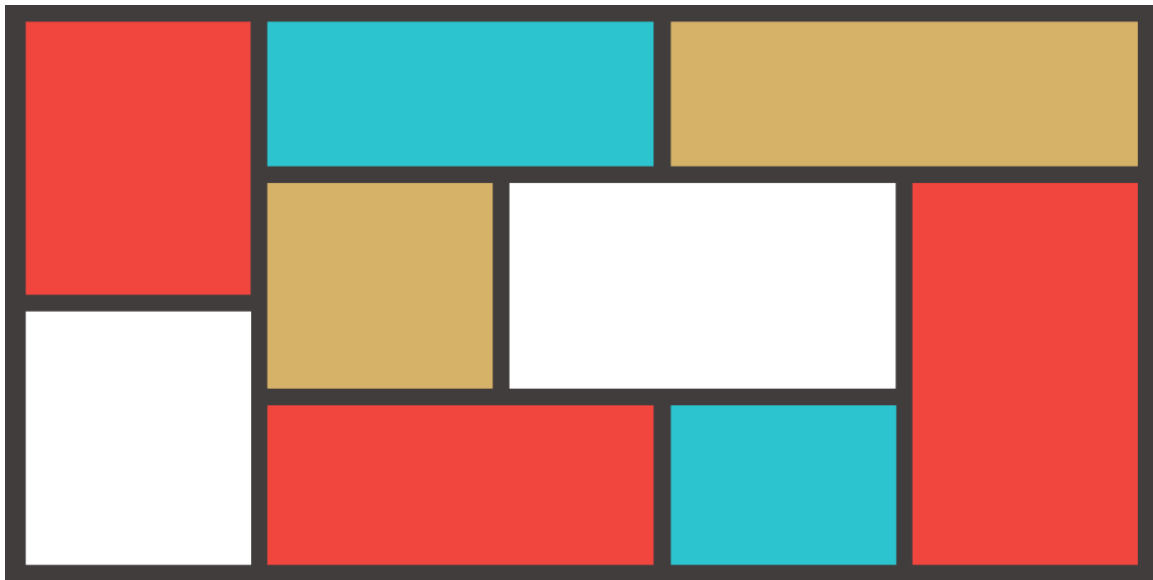
# Codificação Para Front-end

14

---

2026

SENAI - CRTI



## Layout Moderno II: CSS Grid e Responsividade

Nosso e-commerce já tem uma base sólida com Flexbox. Agora é hora de dar o próximo passo: implementar layouts mais estruturados com **CSS Grid** e tornar o site verdadeiramente responsivo com **Media Queries**.

Situação de Aprendizagem

# O Desafio da Aula

## Contexto do Projeto

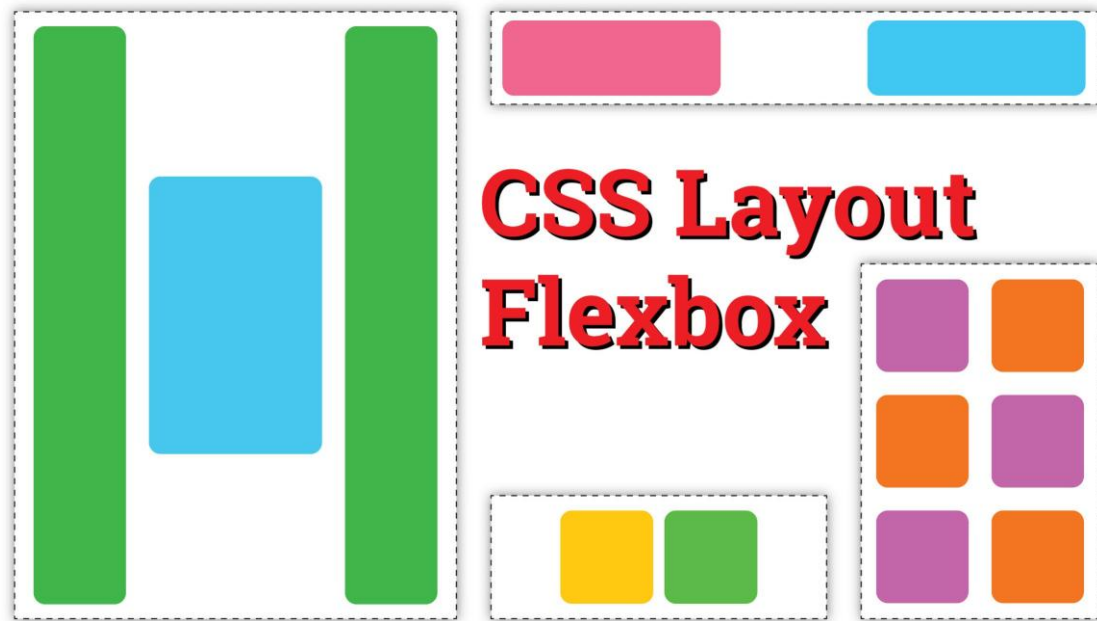
Seu e-commerce já conta com um layout moderno construído com Flexbox. Porém, ao testar em um celular ou tablet, o site quebra, elementos se sobrepõem e a experiência do usuário fica comprometida.

Sua missão nesta aula é **evoluir o projeto**, tornando-o adaptável a qualquer tamanho de tela.

O que você vai aprender

- Criar layouts estruturados com **CSS Grid**
- Aplicar **Media Queries** para responsividade
- Adotar a estratégia **Mobile-First**
- Combinar Grid + Flexbox de forma inteligente
- Testar e validar em diferentes dispositivos

# Retomada: O que já construímos com Flexbox



## O que o Flexbox faz bem

Na aula anterior, usamos Flexbox para criar menus, alinhar itens horizontalmente e organizar componentes em uma única dimensão (linha ou coluna). Ele é excelente para esses casos.

## Onde o Flexbox tem limitações

O Flexbox controla o layout em **um eixo por vez**. Quando precisamos de um grid bidimensional — linhas e colunas ao mesmo tempo — como uma grade de produtos, ele começa a mostrar suas limitações.

```
.menu {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 1rem;  
}
```

# O Problema: Site Não Responsivo

Veja o que acontece quando o e-commerce é acessado em um dispositivo móvel sem responsividade implementada:

## No Celular

Os produtos ficam espremidos ou com scroll horizontal. O texto fica ilegível e os botões impossíveis de clicar.

## No Tablet

O layout para desktop é forçado em uma tela menor, gerando espaços mal aproveitados e elementos desproporcionais.

## No Computador

Funciona bem, mas sem responsividade os outros dispositivos sofrem — e hoje mais de **60% do tráfego web é mobile**.

 Um site não responsivo perde usuários, posicionamento no Google e credibilidade. Responsividade não é opcional — é essencial.

# O que é CSS Grid?

CSS Grid é um sistema de layout **bidimensional** nativo do CSS. Ele permite organizar elementos em linhas e colunas ao mesmo tempo, com controle preciso sobre o posicionamento e o tamanho de cada célula da grade.

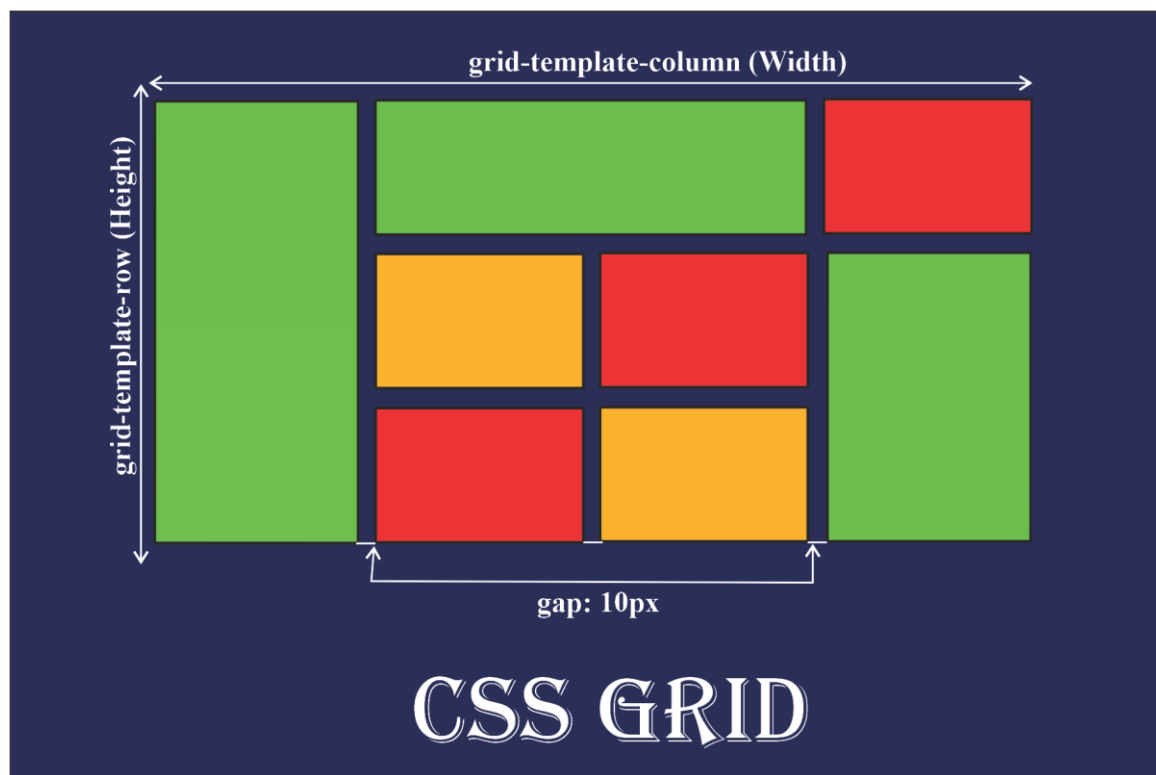
Como ativar o Grid

```
/* Ativar o CSS Grid */  
.container {  
  display: grid;  
}
```

Ao aplicar `display: grid` em um elemento, todos os seus filhos diretos tornam-se itens do grid automaticamente.

A metáfora da grade

Pense em uma planilha do Excel: você define quantas colunas e linhas existem, e então "encaixa" os elementos nas células. Simples assim — mas extremamente poderoso.



# Definindo Colunas e Linhas

As propriedades `grid-template-columns` e `grid-template-rows` definem a estrutura da grade. Você pode usar pixels, porcentagem ou a poderosa unidade `fr`.

## grid-template-columns

```
/* 3 colunas de tamanho fixo */
.grid {
  display: grid;
  grid-template-columns: 200px 200px 200px;
}

/* 3 colunas com fr */
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

## grid-template-rows

```
/* Definindo altura das linhas */
.grid {
  display: grid;
  grid-template-rows: 100px auto 50px;
}

/* Header, conteúdo e footer */
.layout {
  grid-template-rows: 80px 1fr 60px;
}
```

# Unidades Fracionárias: **fr**

O que é a unidade **fr**?

A unidade **fr** representa uma **fração do espaço disponível** no grid. Ela distribui o espaço restante de forma proporcional, tornando o layout flexível e adaptável sem precisar calcular porcentagens manualmente.

É como dividir uma pizza: **1fr 1fr 1fr** dá a cada coluna um terço da pizza. **2fr 1fr** dá ao primeiro o dobro do segundo.

Exemplos práticos

```
/* Três colunas iguais */  
grid-template-columns: 1fr 1fr 1fr;
```

```
/* Sidebar + conteúdo principal */  
grid-template-columns: 1fr 3fr;
```

```
/* Sidebar fixa + conteúdo flexível */  
grid-template-columns: 250px 1fr;
```

```
/* Com a função repeat() */  
grid-template-columns: repeat(3, 1fr);  
/* Equivale a: 1fr 1fr 1fr */
```

```
/* 4 colunas iguais */  
grid-template-columns: repeat(4, 1fr);
```

**Dica:** A unidade **fr** só distribui o espaço *disponível*, ou seja, o que sobrou após colunas com tamanho fixo (px) serem calculadas. Isso a torna ideal para layouts híbridos.

# A Função `repeat()`

A função `repeat()` evita repetição no código e torna as definições de grid mais legíveis. Ela aceita um número de repetições e o tamanho de cada trilha.

## Sintaxe básica

```
repeat(n, tamanho)

/* n = quantas vezes
repetir */
/* tamanho = tamanho de
cada trilha */
```

## auto-fill

```
/* Cria o máximo de colunas
que couberem na largura
*/
grid-template-columns:
  repeat(auto-fill,
    minmax(200px, 1fr));
```

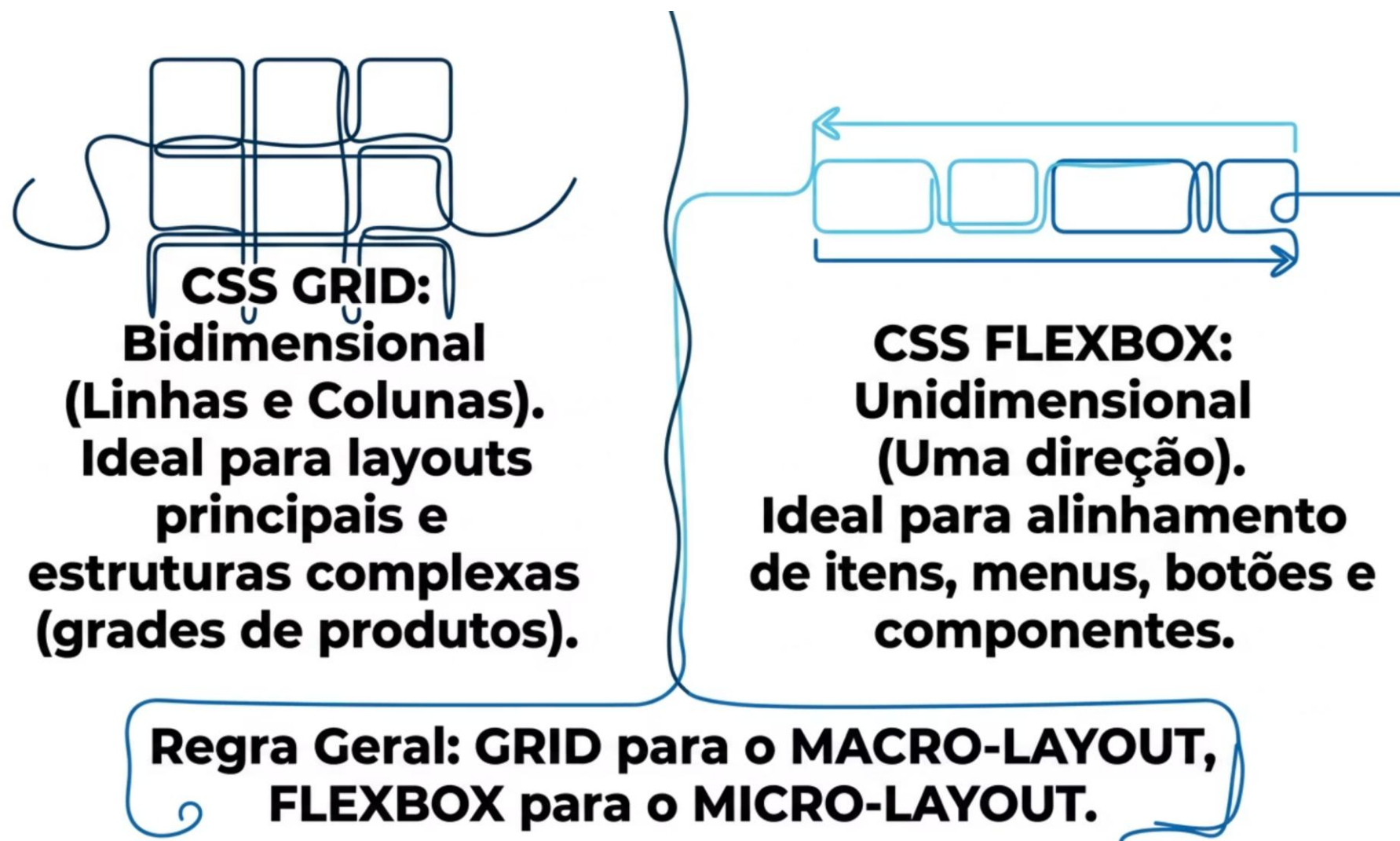
## auto-fit

```
/* Expande colunas para
preencher o espaço */
grid-template-columns:
  repeat(auto-fit,
    minmax(150px, 1fr));
```

✅ **Dica avançada:** `repeat(auto-fill, minmax(200px, 1fr))` cria grids *intrinsecamente responsivos* sem precisar de media queries — as colunas se ajustam automaticamente!

# Grid vs Flexbox: Quando Usar Cada Um?

Essa é uma das dúvidas mais comuns. A boa notícia: **eles se complementam** e podem — e devem — ser usados juntos no mesmo projeto.



→ Use Grid para  
Grades de produtos, layout principal da página, galerias de imagens, qualquer estrutura que precise de controle em 2 dimensões.

→ Use Flexbox para  
Menu de navegação, alinhamento de ícones e botões, centralizar conteúdo, agrupar itens em linha com espaçamento uniforme.

# Introdução às Media Queries

Media Queries são a ferramenta do CSS que permite aplicar estilos diferentes dependendo das características do dispositivo — principalmente a **largura da tela**. Com elas, o mesmo site se comporta de forma diferente em celular, tablet e desktop.



# Sintaxe das Media Queries

## Estrutura básica

```
@media (condição) {  
  /* Estilos aplicados apenas  
    quando a condição for verdadeira */  
  seletor {  
    propriedade: valor;  
  }  
}  
  
/* Exemplo: telas até 768px */  
@media (max-width: 768px) {  
  .produtos-grid {  
    grid-template-columns: 1fr;  
  }  
}
```

## Breakpoints mais comuns

1



Mobile

até 767px

2



Tablet

768px – 1023px

3



Desktop

1024px ou mais

# Estratégia Mobile-First

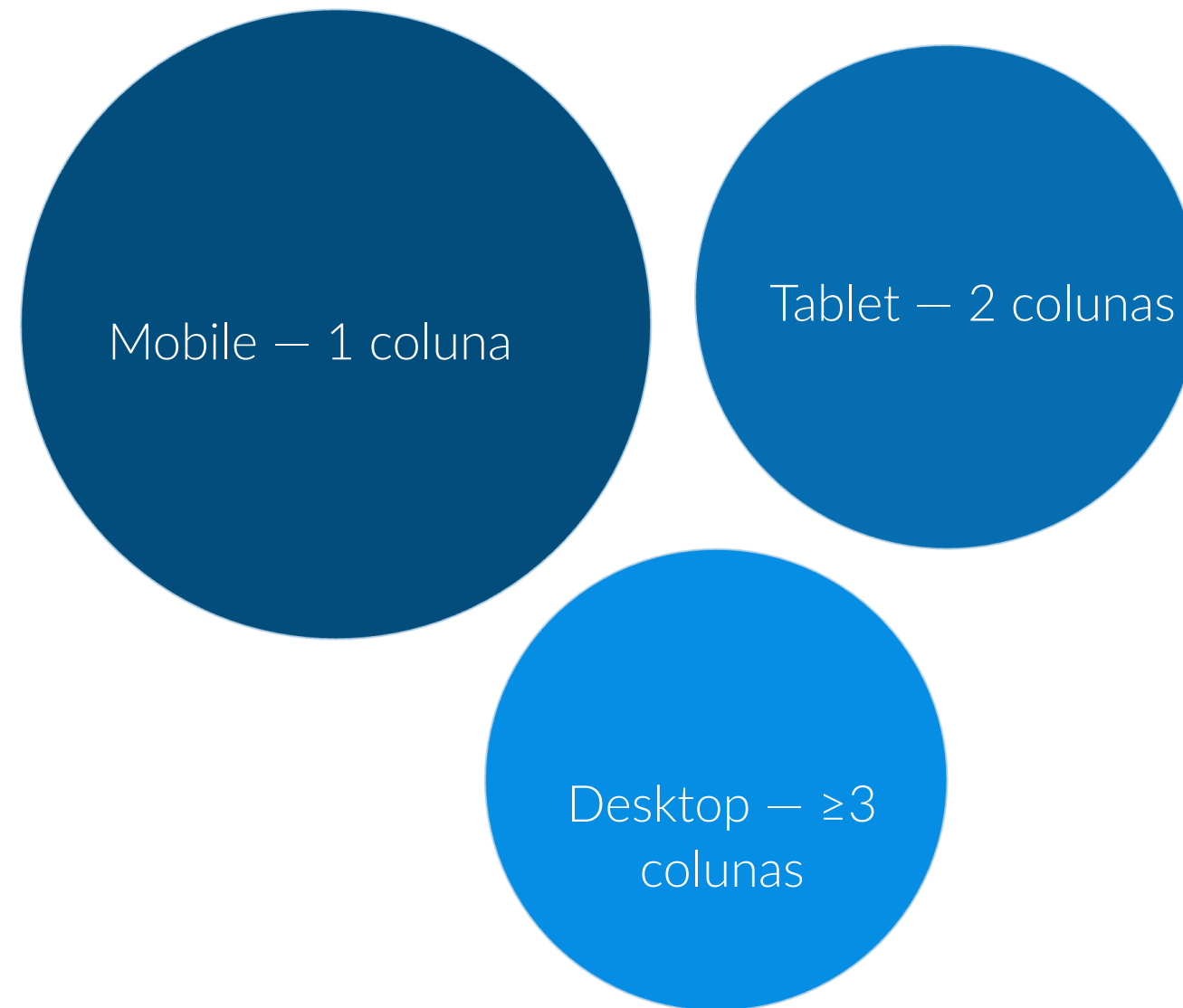
Por que Mobile-First?

A abordagem Mobile-First parte do princípio de que o design deve ser pensado **primeiro para telas menores** e depois expandido para telas maiores.

Isso resulta em código mais limpo, performance melhor em mobile e alinhamento com a realidade: a maioria dos usuários acessa pelo celular.



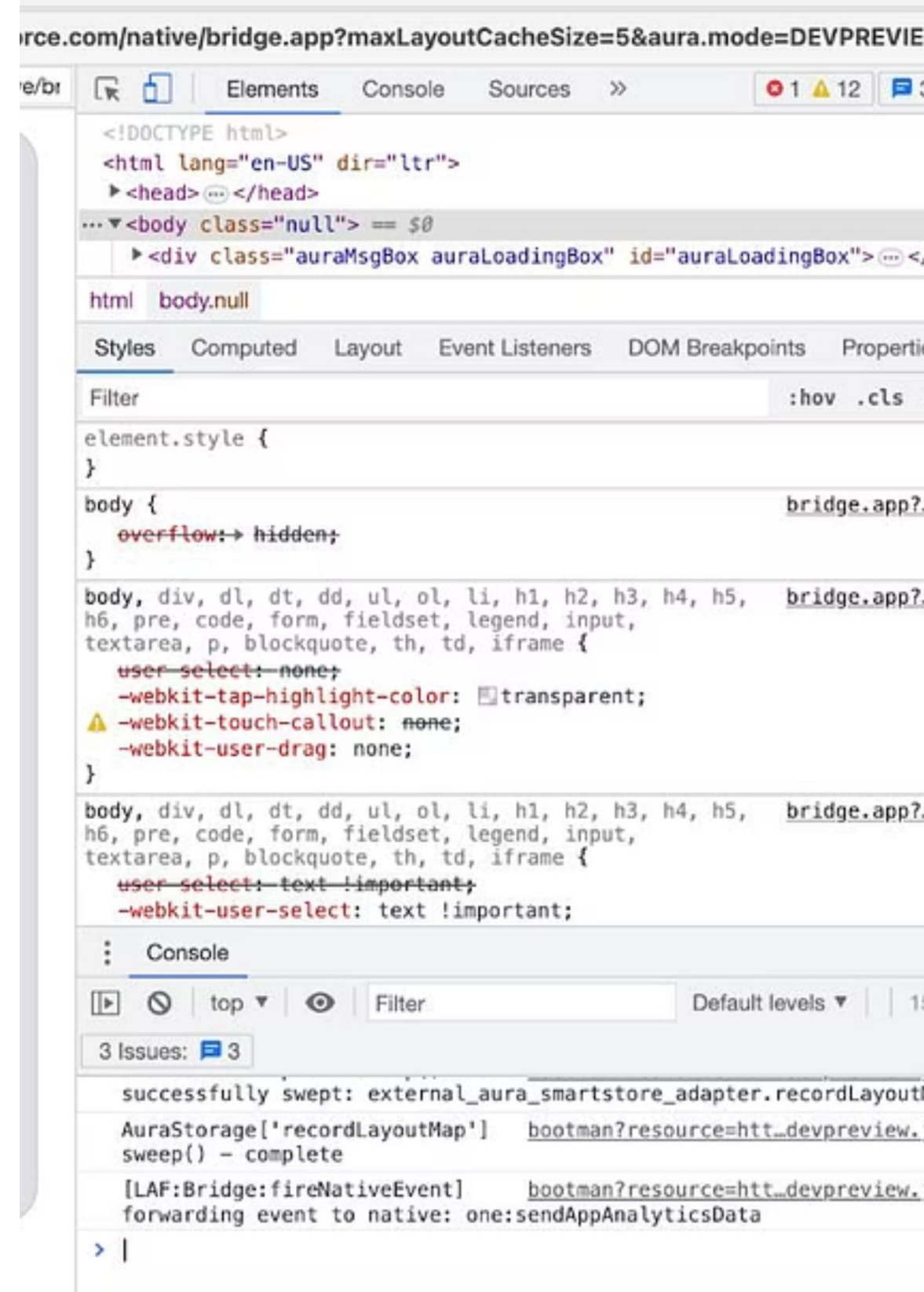
# Breakpoints e Ajustes de Layout



A lógica Mobile-First usa `min-width` nas media queries, o que significa: "a partir desta largura, aplique esses estilos". Os estilos vão sendo **adicionados progressivamente**, nunca sobrescritos desnecessariamente.

# Testando a Responsividade

Antes de considerar o trabalho concluído, é essencial testar o layout em diferentes tamanhos de tela. O Chrome DevTools oferece ferramentas poderosas para isso.



# Como Testar no Navegador

01

---

## Abrir o DevTools

Pressione **F12** ou **Ctrl+Shift+I** no Chrome/Edge para abrir as ferramentas do desenvolvedor.

03

---

## Testar diferentes dispositivos

Use o menu dropdown para selecionar dispositivos como iPhone, iPad ou Galaxy e veja como seu layout se comporta.

02

---

## Ativar o modo responsivo

Clique no ícone de dispositivos (**Ctrl+Shift+M**) para ativar o modo de visualização responsiva.

04

---

## Ajustar manualmente

Arraste a borda da janela para redimensionar livremente e observe os breakpoints sendo ativados em tempo real.

# Checklist de Responsividade

Antes de entregar o projeto, verifique cada item desta lista:

## ☒ Layout e Grid

- ☒ Grade de produtos usa CSS Grid
- ☐ 1 coluna no mobile (até 767px)
- ☐ 2 colunas no tablet (768px–1023px)
- ☐ 3+ colunas no desktop (1024px+)
- ☐ Flexbox mantido no header/menu

## ☒ Usabilidade Mobile

- ☐ Textos legíveis sem zoom
- ☐ Botões com área de toque suficiente
- ☐ Imagens não estourando o layout
- ☐ Sem scroll horizontal indesejado
- ☐ Espaçamentos adequados em mobile



# Atividade Prática

MSEP — APRENDIZAGEM BASEADA EM PROJETOS

Agora é a sua vez! Aplique tudo que aprendeu evoluindo o e-commerce do projeto.

1

Transformar a grade de produtos

Substitua o CSS atual da seção de produtos por CSS Grid com a abordagem Mobile-First (1→2→3 colunas).

2

Implementar Media Queries

Adicione os três breakpoints (mobile, tablet, desktop) com `min-width` e ajuste colunas, espaçamentos e tamanhos de fonte.

3

Testar no DevTools

Abra o modo responsivo do navegador e teste em pelo menos 3 dispositivos diferentes: iPhone, iPad e Desktop 1280px.

4

Validar e entregar

Use o checklist da aula, documente os breakpoints no código com comentários e faça o commit no repositório do projeto.

# Parabéns! Seu E-Commerce Ficou Responsivo

Você dominou CSS Grid, Media Queries e a estratégia Mobile-First. Seu projeto evoluiu de uma página estática para um e-commerce moderno e adaptável a qualquer dispositivo.

## O que você aprendeu

CSS Grid, unidade fr, repeat(), diferença Grid×Flexbox, Media Queries e abordagem Mobile-First.

## O que você construiu

Grade de produtos responsiva: 1 coluna (mobile), 2 colunas (tablet) e 3+ colunas (desktop).

## Próximos passos

Animações CSS, interatividade com JavaScript e integração com APIs para tornar o e-commerce funcional.

 Dúvidas? Revise os slides, consulte a documentação no **MDN Web Docs** ([developer.mozilla.org](https://developer.mozilla.org)) e pratique! A repetição é o caminho do aprendizado.



